

PROGRAMMATION FONCTIONNELLE 2 : PARTIEL 1
mars 2026 — Pierre Rousselin

Exercice 1 : Preuves de logique propositionnelle de base, sur 12 points

Dans tout cet exercice, A , B et C sont trois propositions (en Rocq, des types de sorte `Prop`) quelconques. Dans chacun des cas suivants, donner :

- une preuve Rocq en programmation fonctionnelle, c'est-à-dire avec un seul terme de preuve dans `exact` ;
- une preuve Rocq avec des tactiques de plus haut niveau et `exact` seulement à la fin sur une hypothèse et
- un arbre de dérivation en déduction naturelle.

1. (4 points) `Lemma and_assoc (h : A /\ (B /\ C)) : (A /\ B) /\ C.`
2. (4 points) `Lemma or_comm (h : A \/ B) : B \/ A.`
3. (4 points) `Lemma imp_imp_and (h : A -> (B -> C)) : A /\ B -> C.`

--- * ---

Exercice 2 : Typage de fonctions, sur 13 points

On considère la fonction OCaml suivante :

```
let dup f x = f x x
```

On rappelle que la notation `( op )` en OCaml désigne la fonction (préfixe) associée à l'opérateur infixe `op`. Par exemple, `( + ) 3 2` vaut 5.

1. (1 point) Quelle est la valeur de `dup ( + ) 3` ? de `dup ( && ) true` ?
2. (1 point) Quelle est la valeur de `dup ( + )` ? Comment appelle-t-on cette fonction en langage naturel ?
3. (1 point) Quelle est la valeur de `dup ( && )` ? À quelle autre fonction plus simple est-elle égale ?
4. (1 point) Écrire une définition de `dup` sans aucun sucre syntaxique (avec le mot-clé `fun`).
5. (2 points) Donner le type de `dup`.

On considère maintenant la fonction OCaml suivante :

```
let const c x = c
```

6. (1 point) Quelle est la valeur de `const 42 true` ?
7. (1 point) Quelle est la valeur de `const (fun x -> x + 1) true 10` ?
8. (1 point) Quel est le type de `const (fun x -> x + 1)` ?
9. (1 point) Quelle est la valeur de `const ( + ) true 4 2` ? de `const ( && ) 42 true false` ?
10. (1 point) Quel est le type de `const` ?
11. (1 point) Quelle est la valeur de `dup (const (fun x -> x + 1)) 42` ?
12. (1 point) Que dire (type et valeur) de `fun f -> dup (const f)` ?

--- * ---

Exercice 3 : Entiers naturels, sur 13 points

La définition du type `nat` des entiers naturels en Rocq est rappelée en fin d'énoncé. On définit les fonctions `double` et `half` de la façon suivante :

```

Fixpoint double (n : nat) :=
  match n with
  | 0 => 0
  | S p => S (S (double p))
  end.
Fixpoint half (n : nat) :=
  match n with
  | 0 | 1 => 0
  | S (S p) => S (half p)
  end.

```

1. (1 point) Quelle est la preuve commune des 5 lemmes suivants ?


```

Lemma half_0 : half 0 = 0.
Lemma half_1 : half 1 = 0.
Lemma half_succ_succ (n : nat) : half (S (S n)) = S (half n).
Lemma double_0 : double 0 = 0.
Lemma double_succ (n : nat) : double (S n) = S (S (double n)).

```
2. (2 point) En justifiant chaque égalité avec un des lemmes ci-dessus, évaluer pas à pas :
 - a) `double 2` (c'est-à-dire `double (S (S 0))`)
 - b) `half 5` (c'est-à-dire `half (S (S (S (S (S 0)))))`)
3. (2 point) Donner la preuve en Rocq, puis la « preuve papier très détaillée » (où chaque égalité est justifiée) du lemme suivant :


```

Lemma half_double (n : nat) : half (double n) = n.

```
4. (1 point) Que dire, pour un `n` de type `nat`, de `double (half n)` ? (Une preuve formelle n'est pas demandée ici.)
5. (1 point) On rappelle les propriétés suivantes de l'addition :


```

Lemma add_0_l (n : nat) : 0 + n = n.
Lemma add_0_r (n : nat) : n + 0 = n.
Lemma add_succ_l (n m : nat) : S n + m = S (n + m).
Lemma add_succ_r (n m : nat) : n + S m = S (n + m).

```

 Parmi celles-ci, lesquelles ont besoin d'une preuve par induction ?
6. (2 points) Donner la preuve en Rocq et la « preuve papier très détaillée » (où chaque égalité est justifiée) du lemme suivant :


```

Lemma double_eq_add_diag (n : nat) : double n = n + n.

```
7. (1 point) En déduire la preuve « papier très détaillée » de :


```

Lemma half_add_diag (n : nat) : half (n + n) = n.

```
8. (1 point) La multiplication des entiers naturelle est rappelée en fin d'énoncé. On considère l'énoncé suivant, avec un début de preuve :


```

Lemma twice_eq_add_diag (n : nat) : 2 * n = n + n.
Proof.
  simpl.

```

 Quel est le but après la tactique `simpl.` ? Écrire les étapes successives de réduction. Donner la fin de la preuve en Rocq.
9. (1 point) Donner la définition en Rocq de la fonction `two_pow`, de `nat` vers `nat`, qui à un entier naturel n associe 2^n .
10. (1 point) Que dire, pour un `n` de type `nat` de `half (two_pow (S n))` ? En donner une preuve papier très détaillée.

--- * ---

Exercice 4 : Sur les listes, sur 10 points

La définition des listes est rappelée en fin d'énoncé, avec les fonctions `app` (notation : `l1 ++ l2` pour `app l1 l2`) et `length`. Dans tout l'exercice, `A` désigne un type quelconque. On considère la fonction `rev_append` définie ci-dessous :

```
Fixpoint rev_append (l1 l2 : list A) :=
  match l1 with
  | [] => l2
  | h :: t => rev_append t (h :: l2)
end.
```

Dans tout cet exercice, vous pourrez utiliser les lemmes de l'exercice précédent et les lemmes suivants (conséquences directes des définitions) :

```
Lemma app_nil_l (l : list A) : [] ++ l = l.
Lemma app_cons_l (h : A) (t l : list A) : (h :: t) ++ l = h :: (t ++ l).
Lemma length_nil : length (@nil A) = 0.
Lemma length_cons (h : A) (t : list A) : length (h :: t) = S (length t).
Lemma rev_append_nil_l (l : list A) : rev_append [] l = l.
Lemma rev_append_cons_l (h : A) (t l : list A) : rev_append (h :: t) l = rev_append t (h :: l).
```

1. (1 point) Évaluer en donnant toutes les étapes de calcul et en justifiant chaque égalité : `rev_append [3; 4] [1; 2]`.
2. (2 points) Écrire la preuve papier très détaillée de :
`Lemma length_app (l1 l2 : list A) : length (l1 ++ l2) = length l1 + length l2.`
3. (2 points) En raisonnant par induction sur `l1`, **tout en gardant `l2` quelconque** (l'hypothèse d'induction est donc encore universellement quantifiée), écrire la preuve papier très détaillée de :
`Lemma length_rev_append (l1 : list A) :
 forall l2, length (rev_append l1 l2) = length l1 + length l2.`
4. (2 points) Même chose pour :
`Lemma rev_append_rev_append (l1 : list A) :
 forall (l2 l3 : list A), rev_append (rev_append l1 l2) l3 = rev_append l2 (l1 ++ l3).`
5. (1 point) Définir la fonction `rev`, qui renverse une liste, en utilisant `rev_append`.
6. (1 point) Prouver avec une preuve papier très détaillée :
`Lemma rev_rev (l : list A) : rev (rev l) = l.`
7. (1 point) En déduire :
`Lemma rev_inj (l1 l2 : list A) : rev l1 = rev l2 -> l1 = l2.`

--- * ---

Annexe 1 : règles de la déduction naturelle

$$\begin{array}{c}
 \frac{}{\Gamma, A \vdash A} (\text{Ax}) \\
 \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge\text{I}) \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge\text{E-g}) \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge\text{E-d}) \\
 \frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (\vee\text{I-g}) \qquad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (\vee\text{I-d}) \qquad \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash P \quad \Gamma, B \vdash P}{\Gamma \vdash P} (\vee\text{E}) \\
 \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} (\rightarrow\text{I}) \qquad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (\rightarrow\text{E})
 \end{array}$$

$$\frac{\Gamma, x : T \vdash P}{\Gamma \vdash \forall x : T, P} (\forall I)$$

$$\frac{\Gamma \vdash \forall (x : T), P \quad \Gamma \vdash t : T}{\Gamma \vdash P\{x/t\}} (\forall E)$$

Quelques définitions standard en Rocq

```

Inductive bool := true | false.
Inductive and (A B : Prop) : Prop := conj (hA : A) (hB : B).
Inductive or (A B : Prop) : Prop :=
  or_introl (hA : A) | or_intror (hB : B).
Inductive nat : Set := 0 : nat | S : nat -> nat.
(* Notations usuelles : 0 pour 0, 1 pour (S 0), etc *)
Fixpoint add (n m : nat) :=
  match n with
  | 0 => m
  | S p => S (add p m)
  end.
Fixpoint mul (n m : nat) :=
  match n with
  | 0 => 0
  | S p => m + p * m
  end.
Inductive list (A : Type) : Type :=
  nil : list A | cons : A -> list A -> list A.
(* Notation [] pour nil et h :: t pour (cons h t) *)
Fixpoint app {A : Type} (l0 l1 : list A) :=
  match l0 with
  | [] => l1
  | h :: t => h :: (app t l1)
  end.
Fixpoint length {A : Type} (l : list A) :=
  match l with
  | [] => 0
  | _ :: t => S (length t)
  end.

```

Liste de tactiques Rocq vues en cours, TD et TP

- **exact** : terminer la preuve en cours avec un terme de preuve
- **destruct** : raisonner par cas sur une valeur en liant des variables éventuelles aux arguments éventuels du ou des constructeurs
- **left** ou **right** : choisir le premier ou le deuxième constructeur ; plus souvent utilisé pour prouver une disjonction
- **split** : utiliser l'unique constructeur ; plus souvent utilisé pour prouver une disjonction
- **reflexivity** : terminer la preuve en cours lorsque le but se réduit à **a = a**
- **rewrite** : remplacer un terme par un autre qui lui est égal
- **induction** : comme **destruct** mais avec des hypothèses d'induction dans le(s) cas d'argument(s) dont le type est le même que la valeur sur laquelle on raisonne par induction
- **specialize** : donner des arguments à une fonction ou, ce qui revient au même, instancier un lemme.
- **simpl** : calculer en utilisant les définitions
- **intros** : introduire des variables ou hypothèses depuis le but vers le contexte local